CSS3

Cascading Style Sheets

Complete Lecture Notes with Application Examples

- Selectors & Specificity
- Box Model, Margin & Padding
- Typography & Colours
- Backgrounds & Borders
- Flexbox Layout
- Grid Layout
- Positioning & Z-Index
- Transitions, Animations & Transforms
- Navigation Menus & Dropdowns
- Media Queries & Responsive Design

STATIC WEB DEVELOPMENT / FRONT-END • TCHAPGA C. • 2026

Table of Contents

Table of Contents 2
1. Introduction to CSS3 4
1.1 What is CSS3? 4
1.2 The Web Styling Stack 4
1.3 How CSS Rules Work — General Syntax 4
2. Applying CSS to HTML 6
2.1 Inline Style 6
2.2 Internal (Embedded) Style Sheet 6
2.3 External Style Sheet 6
3. Selectors 7
3.1 Basic Selectors 7
3.2 Combinator Selectors 7
3.3 Attribute Selectors 7
3.4 Pseudo-Classes 7
4. Typography — Fonts & Text 9
4.1 Font Family 9
4.2 Font Size 9
4.3 Font Style and Weight 9
4.4 Text Formatting 9
5. Colours in CSS3 11
6. Backgrounds 12
6.1 Background Colour & Image 12
6.2 Background Properties 12
6.3 CSS Gradients 12
7. Borders & Shadows 13
7.1 Border Properties 13
7.2 Border Radius 13
7.3 Box Shadow 13
8. The CSS Box Model 14
8.1 box-sizing 14
8.2 Margin & Padding 14
8.3 Overflow 14
9. Float & Display 16
9.1 Float 16
9.2 Display Property 16
10. Flexbox Layout 17
10.1 Flex Container Properties 17
10.2 Flex Item Properties 17
10.3 Flexbox Application — Responsive Navigation Bar 17

11. CSS Grid Layout 19
11.1 Defining a Grid 19
11.2 Placing Items 19
11.3 Grid Application — Classic Page Layout 19
12. CSS Positioning 21
12.1 Z-Index 21
13. Navigation Bars & Dropdown Menus 22
13.1 Horizontal Navigation Bar 22
14. Transitions, Animations & Transforms 23
14.1 CSS Transitions 23
14.2 CSS Animations with @keyframes 23
14.3 2D Transforms 23
15. Media Queries & Responsive Design 25
15.1 Syntax 25
15.2 Mobile-First Breakpoints 25
15.3 Responsive Card Grid Application 25
16. Complete Mini Application — Student Portal 27
16.1 HTML Structure 27
16.2 CSS (portal.css) 28
17. Quick Reference 31
17.1 Most-Used CSS Properties 31

1. Introduction to CSS3

Cascading Style Sheets (CSS) is a style sheet language used to describe the presentation of a document written in HTML or XML. While HTML defines the structure and content of a web page, CSS controls how it looks — colours, fonts, spacing, layout, and animations.

1.1 What is CSS3?

CSS3 is the latest evolution of CSS, developed and maintained by the W3C (World Wide Web Consortium). It is split into independent modules so that different parts can progress at different speeds:

- Selectors — target elements with precision
- Box Model — margin, padding, border
- Backgrounds and Borders — gradients, radius, shadows
- Text Effects — shadows, overflow, wrapping
- 2D/3D Transformations — rotate, scale, translate
- Animations and Transitions
- Multiple Column Layout
- Flexbox and Grid Layout
- Media Queries — responsive design
- User Interface — resize, box-sizing, outline

CSS3 is fully backwards-compatible with CSS2. Browsers continue to support all CSS2 rules, and CSS3 adds new capabilities on top.

1.2 The Web Styling Stack

Layer	Technology	Role
Structure	HTML5	Defines what content is and means
Presentation	CSS3	Controls appearance: colours, fonts, layout
Behaviour	JavaScript	Adds interactivity and dynamic updates

1.3 How CSS Rules Work — General Syntax

Every CSS rule follows the same pattern: a selector identifies which HTML elements to style, followed by a declaration block containing one or more property-value pairs:

```
selector {
    property: value;
    property: value;
}

/* Example */
h1 {
    color: #1A237E;
    font-size: 2em;
    font-family: Arial, sans-serif;
}
```

Part	Explanation

`selector`	Identifies the HTML element(s) to style (e.g. h1, .class, #id)
`property`	The aspect of the element to change (e.g. color, font-size)
`value`	The setting to apply (e.g. red, 16px, bold)
`/* ... */`	CSS comment — ignored by the browser

2. Applying CSS to HTML

There are three ways to attach CSS rules to an HTML document, each with different scope and priority.

2.1 Inline Style

Applied directly to a single element using the style attribute. Highest specificity — overrides all other methods. Use sparingly; makes maintenance difficult.

```
<p style="color: red; font-size: 18px;">This text is red.</p>
```

2.2 Internal (Embedded) Style Sheet

Defined inside a <style> element in the <head> section. Applies to the single HTML page. Useful for single-page prototypes.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <style>
    body { background-color: #f0f0f0; }
    h1   { color: #1A237E; }
    p    { font-size: 16px; line-height: 1.6; }
  </style>
</head>
<body>
  <h1>Hello World</h1>
  <p>Internal stylesheet example.</p>
</body>
</html>
```

2.3 External Style Sheet

A separate .css file linked with <link> in the <head>. This is the recommended approach for real projects: one file controls the look of the entire website.

```
<!-- index.html -->
<head>
  <link rel="stylesheet" href="css/style.css">
</head>
```

```
/* css/style.css */
body {
    margin: 0;
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #fafafa;
}
h1 { color: #1A237E; }
```

Method	Scope	Priority	Best For
`Inline`	Single element	Highest	Quick one-off overrides
`Internal`	Single page	Medium	Single-page prototypes
`External`	Entire site	Normal	All real projects

3. Selectors

CSS selectors define which HTML elements a rule targets. Mastering selectors is the key to writing clean, maintainable CSS.

3.1 Basic Selectors

```
/* Universal — targets every element */
* { box-sizing: border-box; margin: 0; padding: 0; }

/* Type — targets all <p> elements */
p { color: #333; }

/* Class — targets elements with class="card" */
.card { background: white; border-radius: 8px; }

/* ID — targets the unique element with id="hero" */
#hero { height: 100vh; }

/* Group — same style for multiple selectors */
h1, h2, h3 { font-family: Georgia, serif; }
```

3.2 Combinator Selectors

Combinator	Syntax	Selects
`Descendant`	A B	All B anywhere inside A
`Child`	A > B	B that are direct children of A only
`Adjacent sibling`	A + B	First B immediately after A
`General sibling`	A ~ B	All B that are siblings of A

```
/* All <a> inside <nav> */
nav a { text-decoration: none; color: white; }

/* Direct <li> children of <ul> only */
ul > li { list-style: square; }

/* First <p> immediately after an <h2> */
h2 + p { font-size: 1.1em; font-weight: bold; }

/* All <p> siblings that follow a <div> */
div ~ p { color: gray; }
```

3.3 Attribute Selectors

```
[type="text"]     { border: 1px solid #ccc; }     /* exact match  */
[class^="btn-"]   { padding: 8px 16px; }           /* starts with  */
[href$=".pdf"]    { color: red; }                  /* ends with    */
[alt*="logo"]     { width: 120px; }                /* contains     */
```

3.4 Pseudo-Classes

Pseudo-classes target elements based on state or position, not just their tag/class/id.

Pseudo-class	Triggers When
`:hover`	Mouse is over the element
`:focus`	Element has keyboard/input focus

Pseudo-class	Triggers When
`:active`	Element is being clicked
`:visited`	Link has been visited
`:first-child`	Element is the first child of its parent
`:last-child`	Element is the last child
`:nth-child(n)`	Element is the nth child (e.g. :nth-child(2n) = every even)
`:not(selector)`	Element does NOT match the given selector
`::before`	Virtual element inserted before content
`::after`	Virtual element inserted after content

```
/* Link state ordering — MUST follow L-V-H-A */
a:link    { color: #1A237E; }
a:visited { color: #7B1FA2; }
a:hover   { color: #E65100; text-decoration: underline; }
a:active  { color: #B71C1C; }

/* Style every odd table row */
tr:nth-child(odd)  { background: #E8EAF6; }
tr:nth-child(even) { background: #FFFFFF; }

/* Add decorative quote marks with pseudo-elements */
blockquote::before { content: '\201C'; font-size: 3em; color: #ccc; }
```

4. Typography — Fonts & Text

4.1 Font Family

The font-family property specifies a prioritised list of font names. The browser uses the first available font; the last entry should always be a generic family as a fallback.

```
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
h1, h2 {
    font-family: Georgia, 'Times New Roman', Times, serif;
}
code, pre {
    font-family: 'Courier New', Courier, monospace;
}

/* Google Font (link in <head> first) */
@import url('https://fonts.googleapis.com/css2?family=Roboto&display=swap');
body { font-family: 'Roboto', sans-serif; }
```

4.2 Font Size

Unit	Type	Example	Notes
px	Absolute	font-size: 16px	Fixed size; ignores user browser settings
em	Relative	font-size: 1.5em	Relative to parent element's font size
%	Relative	font-size: 120%	Relative to parent
rem	Relative	font-size: 1rem	Relative to root <html> element — preferred
vw	Viewport	font-size: 4vw	Relative to viewport width

```
html { font-size: 16px; }   /* 1rem = 16px */

body  { font-size: 1rem; }       /* 16px */
h1    { font-size: 2.5rem; }     /* 40px */
h2    { font-size: 2rem; }       /* 32px */
small { font-size: 0.875rem; }   /* 14px */
```

4.3 Font Style and Weight

```
/* Style */
em       { font-style: italic; }
.oblique { font-style: oblique; }
.normal  { font-style: normal; }

/* Weight */
strong  { font-weight: bold; }      /* 700 */
.thin   { font-weight: 300; }
.black  { font-weight: 900; }

/* Variant */
.caps   { font-variant: small-caps; }
```

4.4 Text Formatting

```
/* Decoration */
.underline    { text-decoration: underline; }
.strikethrough{ text-decoration: line-through; }
.overline     { text-decoration: overline; }
a             { text-decoration: none; }   /* remove underline from links */

/* Alignment (block elements only) */
.left    { text-align: left; }
.center  { text-align: center; }
.right   { text-align: right; }
.justify { text-align: justify; }

/* Spacing */
p {
    line-height: 1.6;             /* 1.6x the font size */
    letter-spacing: 0.05em;       /* space between letters */
    word-spacing: 0.2em;          /* space between words */
}

/* Transform */
.uppercase { text-transform: uppercase; }
.capitalize{ text-transform: capitalize; }

/* Text shadow */
h1 { text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }
/* syntax: h-offset  v-offset  blur  color */
```

5. Colours in CSS3

CSS3 supports five colour notations. Understanding each is important for professional styling and working with design tools.

Notation	Syntax	Example
Named	color-name	color: tomato
HEX	#RRGGBB	color: #E65100
Short HEX	#RGB	color: #F50 (= #FF5500)
RGB	rgb(r,g,b)	color: rgb(230,81,0)
RGBA	rgba(r,g,b,a)	color: rgba(230,81,0,0.6)
HSL	hsl(h,s%,l%)	color: hsl(25,100%,45%)
HSLA	hsla(h,s%,l%,a)	color: hsla(25,100%,45%,0.8)

```
/* Fully opaque navy */
.navy     { color: #1A237E; }

/* Semi-transparent overlay */
.overlay  { background-color: rgba(0,0,0,0.5); }

/* HSL is great for generating colour palettes */
.base     { background: hsl(210, 100%, 50%); }   /* pure blue */
.light    { background: hsl(210, 100%, 75%); }   /* lighter */
.dark     { background: hsl(210, 100%, 25%); }   /* darker  */
```

6. Backgrounds

6.1 Background Colour & Image

```
/* Solid colour */
body { background-color: #F5F5F5; }

/* Background image */
.hero {
    background-image: url('images/banner.jpg');
    background-size: cover;       /* scale to cover element */
    background-position: center;  /* focal point */
    background-repeat: no-repeat; /* don't tile */
    background-attachment: fixed; /* parallax scroll effect */
}
```

6.2 Background Properties

Property	Values	Effect
`background-repeat`	repeat | no-repeat | repeat-x | repeat-y	Tiles the image
`background-size`	cover | contain | px | %	Scales the image
`background-position`	center | top left | 50% 30%	Positions the image
`background-attachment`	scroll | fixed | local	Scroll behaviour
`background-origin`	border-box | padding-box | content-box	Where bg starts

6.3 CSS Gradients

```
/* Linear gradient */
.banner {
    background: linear-gradient(135deg, #1A237E 0%, #00695C 100%);
}

/* Radial gradient */
.circle {
    background: radial-gradient(circle, #E65100 0%, #1A237E 100%);
}

/* Multiple backgrounds (layered) */
.layered {
    background:
        url('overlay.png') no-repeat center,
        linear-gradient(to bottom, #1A237E, #00695C);
}
```

7. Borders & Shadows

7.1 Border Properties

```
/* Shorthand: width  style  color */
.box  { border: 2px solid #1A237E; }

/* Individual sides */
.card {
    border-top:    4px solid #E65100;
    border-bottom: 1px solid #CCCCCC;
    border-left:   none;
    border-right:  none;
}

/* Border styles */
/* none | solid | dashed | dotted | double | groove | ridge | inset | outset */
.dashed { border: 2px dashed #E65100; }
.double { border: 4px double #1A237E; }
```

7.2 Border Radius

```
/* Rounded corners */
.card   { border-radius: 8px; }
.pill   { border-radius: 999px; }
.circle { border-radius: 50%; }

/* Individual corners: top-left  top-right  bottom-right  bottom-left */
.custom { border-radius: 16px 4px 16px 4px; }
```

7.3 Box Shadow

Syntax: box-shadow: h-offset v-offset blur spread color [inset]

```
/* Soft drop shadow */
.card  { box-shadow: 0 4px 12px rgba(0,0,0,0.15); }

/* Multiple shadows */
.fancy { box-shadow: 0 2px 4px rgba(0,0,0,0.1),
                     0 8px 24px rgba(0,0,0,0.2); }

/* Inset (inner shadow) */
.inset { box-shadow: inset 0 2px 6px rgba(0,0,0,0.2); }
```

8. The CSS Box Model

Every HTML element is represented as a rectangular box. The box model describes how the total size and space of that box is calculated.

```
/*
 *  ┌──────────────────────────────┐   ← margin edge
 *  │          MARGIN              │
 *  │  ┌────────────────────────┐  │   ← border edge
 *  │  │        BORDER          │  │
 *  │  │  ┌──────────────────┐  │  │   ← padding edge
 *  │  │  │     PADDING      │  │  │
 *  │  │  │  ┌────────────┐  │  │  │   ← content edge
 *  │  │  │  │  CONTENT   │  │  │  │
 *  │  │  │  └────────────┘  │  │  │
 *  │  │  └──────────────────┘  │  │
 *  │  └────────────────────────┘  │
 *  └──────────────────────────────┘
 */
```

8.1 box-sizing

By default (content-box), width/height apply to the content only — padding and border are added on top. border-box includes padding and border inside the declared width, which is far easier to work with.

```
/* Modern reset — apply to all elements */
*, *::before, *::after {
    box-sizing: border-box;
}

.box {
    width: 300px;      /* border-box: total width stays 300px */
    padding: 20px;     /* included INSIDE the 300px */
    border: 2px solid; /* included INSIDE the 300px */
}
```

8.2 Margin & Padding

```
/* Shorthand — 1 value: all sides */
.a { margin: 16px; }

/* 2 values: top/bottom  left/right */
.b { margin: 16px 32px; }

/* 4 values: top  right  bottom  left  (clockwise) */
.c { margin: 10px 20px 10px 20px; }

/* Individual sides */
.d {
    margin-top: 24px;
    margin-right: 16px;
    margin-bottom: 24px;
    margin-left: 16px;
}

/* Horizontal centering of a block */
.centered { width: 800px; margin: 0 auto; }

/* Padding works identically */
.card { padding: 24px 32px; }
```

8.3 Overflow

Value	Behaviour
`visible`	Default. Content overflows outside the box border
`hidden`	Clips overflowing content — not accessible
`scroll`	Always shows scroll bars
`auto`	Adds scroll bars only when needed (recommended)

```
.content-box {
    height: 200px;
    overflow: auto;
    word-wrap: break-word;  /* break very long words / URLs */
}
```

9. Float & Display

9.1 Float

float was the original CSS layout technique. An element with float: left/right is taken out of normal flow and placed against the left or right side of its container, with inline content wrapping around it.

```
.image-left {
    float: left;
    margin: 0 16px 16px 0;
    width: 200px;
}

/* Always clear floats after the floating container */
.clearfix::after {
    content: '';
    display: block;
    clear: both;
}

/* Or use clear on the next element */
.footer { clear: both; }
```

9.2 Display Property

Value	Description
`block`	Takes full width, starts on new line (div, p, h1…)
`inline`	Flows in text, no width/height control (span, a, em…)
`inline-block`	Like inline but respects width/height/padding
`none`	Removes element completely from layout (not just hidden)
`flex`	Creates a flex container (Section 10)
`grid`	Creates a grid container (Section 11)

```
/* Make <li> elements sit side-by-side */
nav li { display: inline-block; margin: 0 8px; }

/* Hide element (no space taken) */
.hidden { display: none; }

/* visibility: hidden hides but still takes up space */
.invisible { visibility: hidden; }
```

10. Flexbox Layout

Flexbox (Flexible Box Layout) is a one-dimensional layout model — it arranges items in a single row or column. It is ideal for navigation bars, card rows, centring elements, and any component-level layout.

10.1 Flex Container Properties

```
.container {
    display: flex;

    /* Direction of the main axis */
    flex-direction: row;            /* row | row-reverse | column | column-reverse
*/

    /* Wrapping */
    flex-wrap: wrap;               /* nowrap | wrap | wrap-reverse */

    /* Shorthand for direction + wrap */
    flex-flow: row wrap;

    /* Alignment along the MAIN axis */
    justify-content: space-between; /* flex-start|flex-end|center|space-
around|space-evenly */

    /* Alignment along the CROSS axis */
    align-items: center;            /* flex-start|flex-end|center|stretch|baseline
*/

    /* Alignment of wrapped rows along cross axis */
    align-content: flex-start;      /* same values as justify-content */

    gap: 16px;                      /* space between items */
}
```

10.2 Flex Item Properties

```
.item {
    /* Reorder items visually (default: 0) */
    order: 2;

    /* How much extra space this item absorbs (default: 0) */
    flex-grow: 1;

    /* How much this item shrinks when space is tight (default: 1) */
    flex-shrink: 0;

    /* Initial size before growing/shrinking */
    flex-basis: 200px;

    /* Shorthand: grow  shrink  basis */
    flex: 1 1 200px;

    /* Override container's align-items for this item only */
    align-self: flex-end;
}
```

10.3 Flexbox Application — Responsive Navigation Bar

This example builds a full navigation bar that reflows on mobile using Flexbox:

```
<!-- HTML -->
<nav class="navbar">
  <div class="logo">MySite</div>
```

```
  <ul class="nav-links">
    <li><a href="#">Home</a></li>
    <li><a href="#">About</a></li>
    <li><a href="#">Services</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

```
/* CSS */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #1A237E;
    padding: 0 32px;
    height: 60px;
}
.logo {
    color: white;
    font-size: 1.4rem;
    font-weight: bold;
}
.nav-links {
    display: flex;
    list-style: none;
    gap: 24px;
    margin: 0; padding: 0;
}
.nav-links a {
    color: white;
    text-decoration: none;
    font-size: 0.95rem;
    transition: color 0.2s;
}
.nav-links a:hover { color: #90CAF9; }
```

11. CSS Grid Layout

Grid is a two-dimensional layout system — rows AND columns simultaneously. Use Grid for page-level structure (header, sidebar, main, footer) and anywhere you need precise alignment in both axes.

11.1 Defining a Grid

```
.grid {
    display: grid;

    /* Define 3 columns: 200px | flexible | flexible */
    grid-template-columns: 200px 1fr 1fr;

    /* Define rows (optional — auto-creates rows by default) */
    grid-template-rows: auto;

    /* Shorthand: rows / columns */
    grid-template: auto / 200px 1fr 1fr;

    /* Gap between cells */
    gap: 16px;                  /* row-gap: 16px; column-gap: 16px; */

    /* Alignment of items inside cells */
    justify-items: stretch;     /* start|end|center|stretch */
    align-items: stretch;
}

/* Repeat helper — 3 equal columns */
.cards { grid-template-columns: repeat(3, 1fr); }

/* Auto-fill: as many columns as fit at min 200px */
.gallery { grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); }
```

11.2 Placing Items

```
/* Span across columns */
.header { grid-column: 1 / -1; }     /* from col 1 to last column */
.wide   { grid-column: 2 / 4; }      /* columns 2 and 3 */
.tall   { grid-row: 1 / 3; }         /* rows 1 and 2 */

/* Using named areas */
.layout {
    display: grid;
    grid-template-areas:
        'header  header '
        'sidebar main   '
        'footer  footer ';
    grid-template-columns: 220px 1fr;
    grid-template-rows: 60px 1fr 50px;
    min-height: 100vh;
    gap: 0;
}
header { grid-area: header; }
aside  { grid-area: sidebar; }
main   { grid-area: main; }
footer { grid-area: footer; }
```

11.3 Grid Application — Classic Page Layout

```
<!-- HTML -->
<div class="layout">
  <header>Header</header>
  <aside>Sidebar</aside>
```

```
  <main>Main Content</main>
  <footer>Footer</footer>
</div>
```

```
/* CSS */
.layout {
    display: grid;
    grid-template-areas:
        'header  header'
        'sidebar main  '
        'footer  footer';
    grid-template-columns: 240px 1fr;
    grid-template-rows: 64px 1fr 50px;
    min-height: 100vh;
    gap: 0;
}
header  { grid-area: header;  background: #1A237E; color: white; }
aside   { grid-area: sidebar; background: #E8EAF6; padding: 24px; }
main    { grid-area: main;    padding: 32px; }
footer  { grid-area: footer;  background: #333; color: white; text-align: center; }
```

12. CSS Positioning

The position property removes elements from the normal document flow and places them precisely. Offset properties (top, right, bottom, left) only work when position is set.

Value	Description
`static`	Default. Follows normal document flow. Offsets ignored.
`relative`	Offset from its own normal position. Still occupies original space.
`absolute`	Positioned relative to the nearest positioned ancestor (non-static). Removed from flow.
`fixed`	Positioned relative to the viewport. Always visible; removed from flow.
`sticky`	Hybrid: relative until a scroll threshold, then becomes fixed.

```
/* Absolute positioned badge on a card */
.card    { position: relative; }   /* establishes containing block */
.badge   {
    position: absolute;
    top: -8px;
    right: -8px;
    background: red;
    border-radius: 50%;
    width: 20px; height: 20px;
}

/* Fixed navigation bar */
.navbar {
    position: fixed;
    top: 0; left: 0; right: 0;
    z-index: 1000;
    height: 60px;
}
body { padding-top: 60px; }  /* prevent content hiding under navbar */

/* Sticky table header */
thead th {
    position: sticky;
    top: 0;
    background: white;
    z-index: 10;
}
```

12.1 Z-Index

z-index controls the stacking order of positioned elements. Higher values appear on top. z-index only works on elements with a position other than static.

```
.modal-overlay { position: fixed; inset: 0; z-index: 900; }
.modal-dialog  { position: fixed; z-index: 901; }
.tooltip       { position: absolute; z-index: 500; }
```

13. Navigation Bars & Dropdown Menus

13.1 Horizontal Navigation Bar

```
<!-- HTML -->
<nav class="topnav">
  <ul>
    <li><a href="#">Home</a></li>
    <li class="dropdown">
      <a href="#">Courses &#9660;</a>
      <div class="dropdown-content">
        <a href="#">HTML5</a>
        <a href="#">CSS3</a>
        <a href="#">JavaScript</a>
      </div>
    </li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
```

```
/* CSS */
.topnav ul {
    display: flex;
    list-style: none;
    margin: 0; padding: 0;
    background: #1A237E;
}
.topnav a {
    display: block;
    padding: 16px 20px;
    color: white;
    text-decoration: none;
    transition: background 0.2s;
}
.topnav a:hover { background: #303F9F; }

/* Dropdown container */
.dropdown { position: relative; }

/* Dropdown panel — hidden by default */
.dropdown-content {
    display: none;
    position: absolute;
    top: 100%; left: 0;
    background: white;
    min-width: 180px;
    box-shadow: 0 4px 16px rgba(0,0,0,0.15);
    border-radius: 0 0 6px 6px;
    z-index: 100;
}
.dropdown-content a {
    color: #333;
    padding: 12px 20px;
    border-bottom: 1px solid #eee;
}
.dropdown-content a:hover { background: #E8EAF6; color: #1A237E; }

/* Show on hover */
.dropdown:hover .dropdown-content { display: block; }
```

14. Transitions, Animations & Transforms

14.1 CSS Transitions

Transitions animate a CSS property from one value to another when a state changes (e.g. on :hover).

```
/* Syntax: property  duration  timing-function  delay */
.btn {
    background: #1A237E;
    color: white;
    padding: 12px 24px;
    border-radius: 6px;
    border: none;
    cursor: pointer;
    transition: background 0.3s ease, transform 0.2s ease;
}
.btn:hover {
    background: #303F9F;
    transform: translateY(-2px);  /* lifts the button */
}

/* Timing functions */
/* ease | linear | ease-in | ease-out | ease-in-out | cubic-bezier(...) */
```

14.2 CSS Animations with @keyframes

Animations allow multi-step, auto-playing CSS effects without JavaScript.

```
/* Define the animation */
@keyframes slideIn {
    from { transform: translateX(-100%); opacity: 0; }
    to   { transform: translateX(0);    opacity: 1; }
}

@keyframes pulse {
    0%   { transform: scale(1); }
    50%  { transform: scale(1.05); }
    100% { transform: scale(1); }
}

/* Apply the animation */
.sidebar {
    animation-name: slideIn;
    animation-duration: 0.4s;
    animation-timing-function: ease-out;
    animation-delay: 0s;
    animation-fill-mode: both;   /* keeps end state after animation */
}

/* Shorthand */
.card:hover { animation: pulse 0.6s ease infinite; }
```

14.3 2D Transforms

Function	Example	Effect
`translate()`	transform: translate(50px, 20px)	Move horizontally and vertically
`translateX/Y`	transform: translateX(-100%)	Move along one axis
`scale()`	transform: scale(1.2)	Zoom in/out (1 = normal)
`rotate()`	transform: rotate(45deg)	Rotate clockwise
`skew()`	transform: skew(10deg, 5deg)	Shear distortion

Function	Example	Effect
`matrix()`	transform: matrix(...)	Combine all transforms in one

```
/* Common hover effects */
.zoom-card:hover  { transform: scale(1.03); }
.rotate-icon:hover{ transform: rotate(90deg); }
.lift-btn:hover   { transform: translateY(-4px); }

/* Always pair transforms with transition */
.card {
    transition: transform 0.25s ease, box-shadow 0.25s ease;
}
.card:hover {
    transform: translateY(-6px);
    box-shadow: 0 12px 32px rgba(0,0,0,0.18);
}
```

15. Media Queries & Responsive Design

Media queries allow different CSS rules based on device characteristics (width, height, orientation, resolution). This is the foundation of responsive web design.

15.1 Syntax

```
/* Apply only when viewport is at least 768px wide */
@media (min-width: 768px) {
    .container { max-width: 960px; margin: 0 auto; }
}

/* Apply only on screens narrower than 480px */
@media (max-width: 480px) {
    body { font-size: 14px; }
    .navbar ul { flex-direction: column; }
}

/* Range: tablet */
@media (min-width: 481px) and (max-width: 1024px) {
    .grid { grid-template-columns: repeat(2, 1fr); }
}

/* Print */
@media print {
    nav, footer { display: none; }
    body { font-size: 12pt; color: black; }
}
```

15.2 Mobile-First Breakpoints

Breakpoint	Min-Width	Target
`xs (default)`	< 480px	Small phones
`sm`	480px	Large phones
`md`	768px	Tablets
`lg`	1024px	Laptops
`xl`	1280px	Desktops
`2xl`	1536px	Large monitors

15.3 Responsive Card Grid Application

```
<!-- HTML -->
<div class="card-grid">
  <div class="card">Card 1</div>
  <div class="card">Card 2</div>
  <div class="card">Card 3</div>
  <div class="card">Card 4</div>
</div>
```

```
/* Mobile first: 1 column */
.card-grid {
    display: grid;
    grid-template-columns: 1fr;
    gap: 16px;
    padding: 16px;
}
.card {
    background: white;
    border-radius: 8px;
```

```
    padding: 24px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

/* Tablet: 2 columns */
@media (min-width: 600px) {
    .card-grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop: 4 columns */
@media (min-width: 1024px) {
    .card-grid { grid-template-columns: repeat(4, 1fr); }
}
```

16. Complete Mini Application — Student Portal

This example combines all CSS3 concepts into a minimal, fully responsive student portal with a navbar, hero section, card grid, and footer — approximately 90 lines of CSS.

16.1 HTML Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Portal</title>
  <link rel="stylesheet" href="portal.css">
</head>
<body>

  <!-- NAVIGATION -->
  <nav class="navbar">
    <div class="logo">EduPortal</div>
    <ul class="nav-links">
      <li><a href="#" class="active">Dashboard</a></li>
      <li><a href="#">Courses</a></li>
      <li><a href="#">Grades</a></li>
      <li><a href="#">Profile</a></li>
    </ul>
  </nav>

  <!-- HERO -->
  <section class="hero">
    <h1>Welcome, Ama!</h1>
    <p>You have 3 upcoming assignments this week.</p>
    <a href="#" class="btn-hero">View Schedule</a>
  </section>

  <!-- CARDS -->
  <main class="card-grid">
    <div class="card">
      <div class="card-icon">📚</div>
      <h3>HTML5 Fundamentals</h3>
      <p>Module 4 of 8 &bull; 72% complete</p>
      <div class="progress-bar"><div class="progress"
style="width:72%"></div></div>
      <a href="#" class="btn">Continue</a>
    </div>
    <div class="card">
      <div class="card-icon">🎨</div>
      <h3>CSS3 & Design</h3>
      <p>Module 6 of 8 &bull; 55% complete</p>
      <div class="progress-bar"><div class="progress"
style="width:55%"></div></div>
      <a href="#" class="btn">Continue</a>
    </div>
    <div class="card">
      <div class="card-icon">⚡</div>
      <h3>JavaScript Basics</h3>
      <p>Module 2 of 10 &bull; 20% complete</p>
      <div class="progress-bar"><div class="progress"
style="width:20%"></div></div>
      <a href="#" class="btn">Continue</a>
    </div>
  </main>

  <!-- FOOTER -->
```

```
  <footer>
    <p>&copy; 2025 EduPortal &bull; Internet Application Programming</p>
  </footer>

</body>
</html>
```

16.2 CSS (portal.css)

```
/* === RESET & BASE === */
*, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

body {
    font-family: 'Segoe UI', Arial, sans-serif;
    background: #F5F5F5;
    color: #333;
    line-height: 1.6;
}

/* === NAVBAR === */
.navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    background: #1A237E;
    padding: 0 32px;
    height: 64px;
    position: sticky;
    top: 0;
    z-index: 100;
    box-shadow: 0 2px 8px rgba(0,0,0,0.3);
}
.logo {
    color: white;
    font-size: 1.4rem;
    font-weight: 800;
    letter-spacing: 0.05em;
}
.nav-links { display: flex; list-style: none; gap: 8px; }
.nav-links a {
    color: rgba(255,255,255,0.8);
    text-decoration: none;
    padding: 8px 16px;
    border-radius: 6px;
    font-size: 0.9rem;
    transition: background 0.2s, color 0.2s;
}
.nav-links a:hover, .nav-links a.active {
    background: rgba(255,255,255,0.15);
    color: white;
}

/* === HERO === */
.hero {
    background: linear-gradient(135deg, #1A237E 0%, #00695C 100%);
    color: white;
    text-align: center;
    padding: 80px 32px;
}
.hero h1 { font-size: 2.5rem; margin-bottom: 12px; }
.hero p  { font-size: 1.1rem; opacity: 0.85; margin-bottom: 28px; }
.btn-hero {
    display: inline-block;
    background: #E65100;
    color: white;
    padding: 14px 32px;
    border-radius: 999px;
    text-decoration: none;
    font-weight: 600;
    transition: background 0.3s, transform 0.2s;
```

```
}
.btn-hero:hover { background: #BF360C; transform: translateY(-2px); }

/* === CARD GRID === */
.card-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
    gap: 24px;
    padding: 40px 32px;
    max-width: 1200px;
    margin: 0 auto;
}
.card {
    background: white;
    border-radius: 12px;
    padding: 28px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.08);
    transition: transform 0.25s ease, box-shadow 0.25s ease;
}
.card:hover {
    transform: translateY(-6px);
    box-shadow: 0 12px 28px rgba(0,0,0,0.15);
}
.card-icon { font-size: 2.5rem; margin-bottom: 12px; }
.card h3   { font-size: 1.1rem; color: #1A237E; margin-bottom: 6px; }
.card p    { font-size: 0.85rem; color: #666; margin-bottom: 16px; }

/* Progress bar */
.progress-bar {
    background: #E8EAF6;
    border-radius: 999px;
    height: 8px;
    margin-bottom: 20px;
    overflow: hidden;
}
.progress {
    height: 100%;
    background: linear-gradient(90deg, #1A237E, #00695C);
    border-radius: 999px;
}

/* Card button */
.btn {
    display: inline-block;
    background: #1A237E;
    color: white;
    padding: 10px 24px;
    border-radius: 6px;
    text-decoration: none;
    font-size: 0.9rem;
    transition: background 0.2s;
}
.btn:hover { background: #303F9F; }

/* === FOOTER === */
footer {
    background: #333;
    color: rgba(255,255,255,0.7);
    text-align: center;
    padding: 24px;
    font-size: 0.85rem;
    margin-top: 40px;
}

/* === RESPONSIVE === */
@media (max-width: 600px) {
    .navbar { padding: 0 16px; }
    .nav-links { gap: 0; }
    .nav-links a { padding: 8px 10px; font-size: 0.8rem; }
    .hero h1 { font-size: 1.8rem; }
    .card-grid { padding: 24px 16px; gap: 16px; }
```

```
}
```

17. Quick Reference

17.1 Most-Used CSS Properties

Property	Common Values	Purpose
`color`	#hex | rgb() | name	Text colour
`background`	colour | image | gradient	Element background
`font-family`	Arial, sans-serif	Font stack with fallbacks
`font-size`	16px | 1rem | 1.5em	Text size
`font-weight`	normal | bold | 100-900	Text weight
`text-align`	left | center | right | justify	Horizontal alignment
`line-height`	1.6 | 24px	Vertical spacing between lines
`margin`	0 | 16px | 0 auto	Space outside border
`padding`	16px | 8px 16px	Space inside border
`border`	2px solid #333	Border shorthand
`border-radius`	8px | 50%	Rounded corners
`width / height`	300px | 50% | auto	Element dimensions
`max-width`	1200px	Caps the width for readability
`display`	block | flex | grid | none	Layout mode
`position`	static | relative | absolute | fixed	Positioning method
`top/right/bottom/left`	0 | 16px | 50%	Position offsets
`z-index`	1 | 100 | 9999	Stacking order
`overflow`	auto | hidden | scroll	Content overflow behaviour
`opacity`	0.5 | 1	Element transparency (0=hidden, 1=fully visible)
`transition`	all 0.3s ease	Smooth property changes
`transform`	scale() | rotate() | translate()	Visual transformations
`box-shadow`	0 4px 12px rgba(0,0,0,0.15)	Drop shadow
`cursor`	pointer | default | not-allowed	Mouse cursor style
`gap`	16px	Space between flex/grid items

References: css-tricks.com • developer.mozilla.org • w3schools.com • flexboxfroggy.com • cssgridgarden.com